

a) backend/app/core/security.py

Password hashing bằng Passlib (bcrypt) và tạo JWT bằng python-jose

```
from datetime import datetime, timedelta, timezone
from typing import Any

from jose import jwt
from passlib.context import CryptContext

from app.core.config import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)


def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)


def create_access_token(subject: str, expires_minutes: int | None = None,
extra_claims: dict[str, Any] | None = None) -> str:
    expire_delta = timedelta(minutes=expires_minutes or
settings.access_token_expire_minutes)
    expires_at = datetime.now(timezone.utc) + expire_delta
    payload: dict[str, Any] = {"sub": subject, "exp": expires_at}
    if extra_claims:
        payload.update(extra_claims)
    return jwt.encode(payload, settings.jwt_secret_key,
algorithm=settings.jwt_algorithm)
```

b) backend/app/api/routes/auth.py. API đăng ký/đăng nhập với validate và xử lý lỗi (email trùng, sai mật khẩu...)

endpoint POST /register

```
@router.post("/register", response_model=AuthResponse,
status_code=status.HTTP_201_CREATED)
async def register(payload: UserCreate, db: AsyncSession = Depends(get_db)) ->
AuthResponse:
    try:
```

```

        user = await create_user(db, payload)
    except ValueError as exc:
        message = str(exc)
        if message == "EMAIL_EXISTS":
            raise HTTPException(status_code=status.HTTP_409_CONFLICT,
detail="Email already exists") from None
        if message == "PHONE_EXISTS":
            raise HTTPException(status_code=status.HTTP_409_CONFLICT,
detail="Phone already exists") from None
        if message == "ROLE_NOT_FOUND":
            raise
HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail="Default
role not found") from None
            raise

        access_token = create_access_token(subject=str(user.id),
extra_claims={"roles": [role.code for role in user.roles]})
        return AuthResponse(access_token=access_token,
user=UserRead.model_validate(user))

```

Và POST /login

```

@router.post("/login", response_model=AuthResponse)
async def login(payload: LoginRequest, db: AsyncSession = Depends(get_db)) ->
AuthResponse:
    user = await get_user_by_identifrier(db, payload.identifier)
    if user is None or not verify_password(payload.password, user.password_hash):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")
    if not user.is_active:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="User
is inactive")

    access_token = create_access_token(subject=str(user.id),
extra_claims={"roles": [role.code for role in user.roles]})
    return AuthResponse(access_token=access_token,
user=UserRead.model_validate(user))

```

c) frontend/store/user.ts

Hàm setSession(),localStorage.setItem('cineai_token', token) "Store JWT token in localStorage"

```
function setSession(user: UserProfile | null, token: string | null) {
  currentUser.value = user
  isAuthenticated.value = Boolean(user && token)
  authToken.value = token

  if (process.client) {
    if (user && token) {
      localStorage.setItem('cineai_user', JSON.stringify(user))
      localStorage.setItem('cineai_token', token)
    } else {
      localStorage.removeItem('cineai_user')
      localStorage.removeItem('cineai_token')
    }
  }

  setAuthToken(token)
}
```

f) POST /auth/register, request + response

phần response trả về, thấy rõ access_token (không thấy password gốc ở đâu cả).

auth

POST /api/v1/auth/register Register

Parameters Cancel

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "email": "user@example.com",
  "phone": "string",
  "full_name": "string",
  "date_of_birth": "2026-07-14",
  "gender": "string",
  "password": "stringst"
}
```

Execute Clear

